

OPPORTUNITY INTELLIGENCE FOR TECHNICAL BUILDERS

Turn public signal into credible outreach.

UpSearch researches the company, isolates a real technical problem, and drafts the first message. The human stays in control.



THE STARTING PROBLEM

Applications begin too broad.

A promising role is not enough. Early-career builders need a specific reason to talk to a specific technical team.

01
Signal is scattered

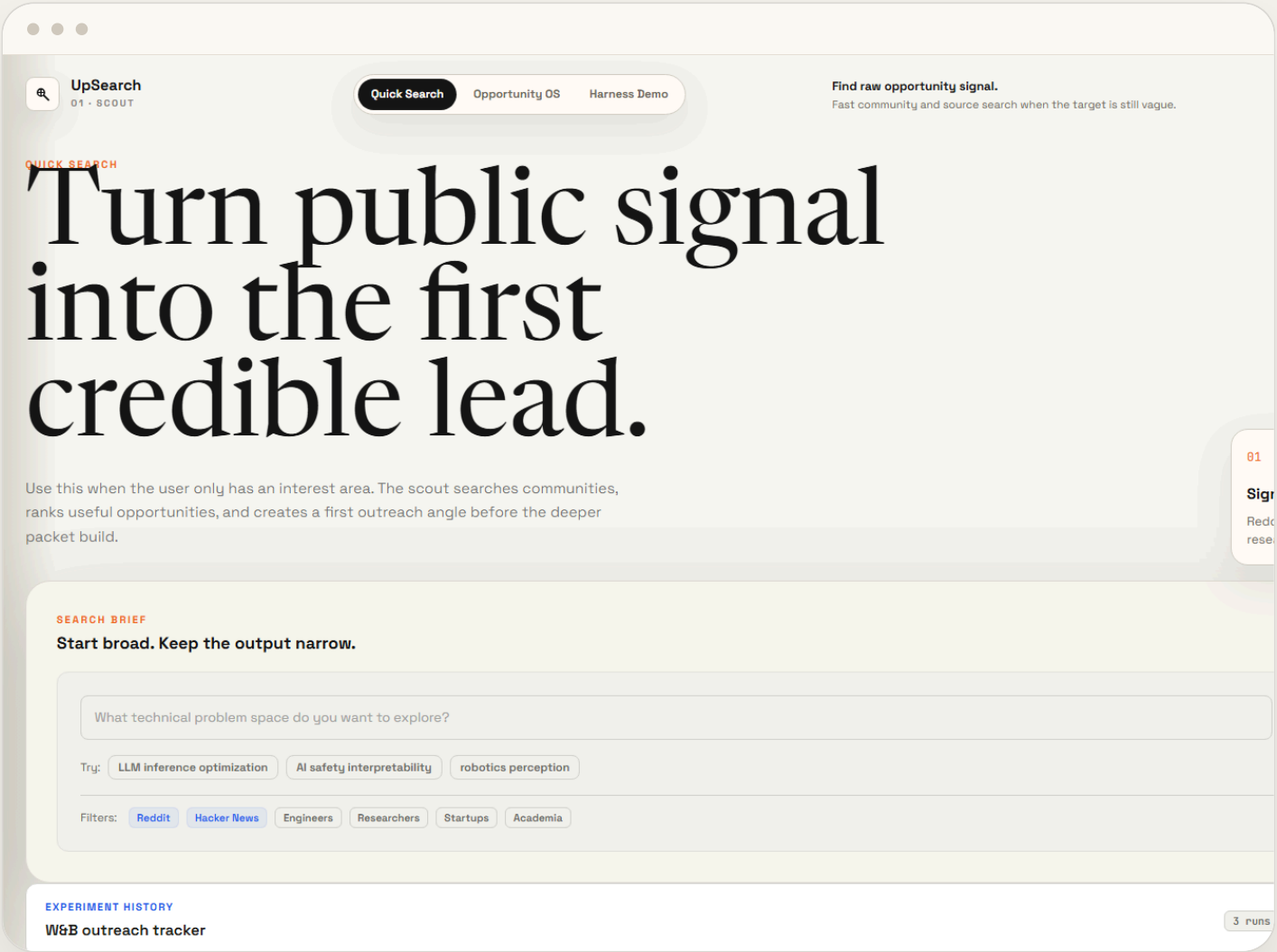
Posts, product updates, engineering blogs, and hiring clues live in different places.

02
Context is expensive

A real outreach angle takes research, synthesis, and a credible adjacent proof point.

03
Volume is the wrong goal

One thoughtful technical conversation can beat another hundred generic applications.



THE PRODUCT LOOP

One company becomes a technical packet.

Opportunity OS moves from a vague target to a company-specific brief, a useful artifact, and outreach worth approving.



01 · Source

Company, roles, public signal

02 · Build

Problem, people, artifact

03 · Verify

Claims, tone, evidence

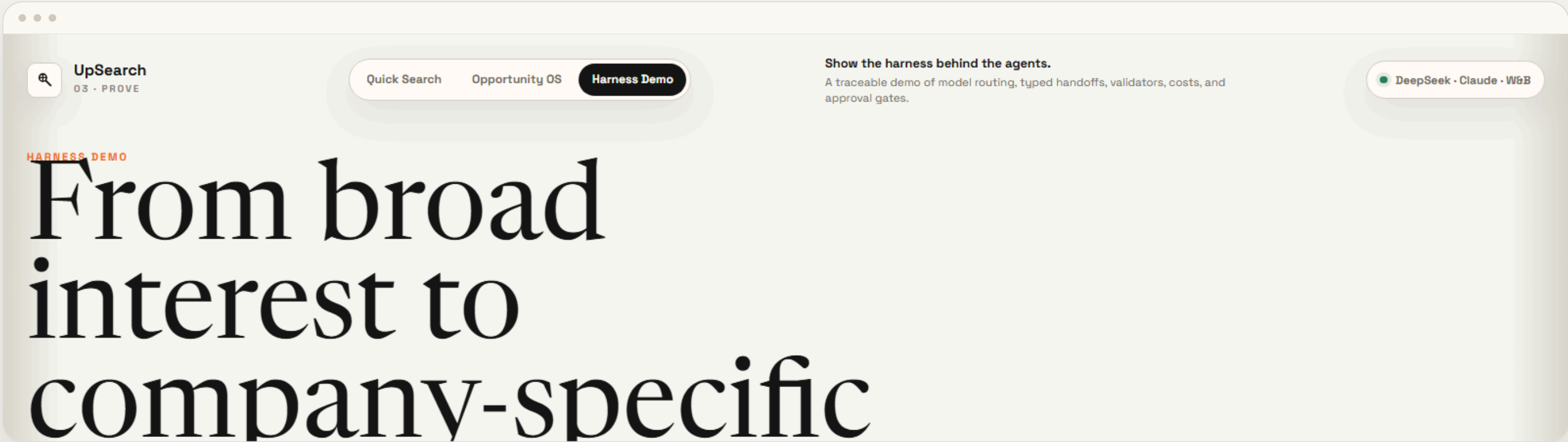
04 · Approve

Exact human action gate

THE COORDINATION LAYER

Agents coordinate. Humans approve.

The harness gives each agent a typed goal, allowed tools, validators, and a ledger event. External actions stay blocked until approval.



Typed handoffs
Each stage leaves structured output.

Supervisor QA
Claims and outreach rules are checked.

Approval gate
No message is sent autonomously.

WHAT THE USER GETS

Useful work before the first message.

The output is not just a draft. It is a compact technical case: the problem, the proof, the artifact, the people, and the outreach variants.

Writes one-page problem brief

06 OUTREACH
Drafts email, LinkedIn note, follow-up

07 QA
Checks claims, sources, word count, tone

ADJACENT PROOF

This student has coursework in AI (CSE 4705) providing foundational understanding of neural network inference, and Convex Optimization (CSE 4820) which directly applies to formulating latency-throughput trade-offs as optimization problems. Their studies in Operating Systems and Computer Organization offer systems-level grounding for reasoning about distributed resource scheduling and GPU memory constraints. While lacking LLM inference runtime experience, they have prototyped optimization heuristics in Python (from AI course projects) and worked with Go, making them capable of building a small-scale scheduling agent on Kubernetes.

OPEN PROBLEMS / 3

Latency and throughput optimization for distributed LLM inference8/18

Serving large language models at scale on a cloud platform requires minimizing latency and maximizing throughput across distributed GPU clusters. This involves optimizing batching strategies, model sharding, and request routing to handle variable loads efficiently.

[Evidence 1](#)

Efficient fine-tuning of open-source LLMs on cloud infrastructure7/18

Fine-tuning large models on cloud platforms involves challenges such as memory management, distributed training orchestration, and cost control. Optimizing these processes is critical for enabling users to customize models without prohibitive resource usage.

[Evidence 1](#)

Real-time data integration for LLM applications6/18

Building LLM applications that work on constantly updating data is a production challenge, requiring frameworks to ingest, process, and serve real-time data streams to models with low latency and high consistency.

[Evidence 1](#) [Evidence 2](#)

PEOPLE MAP / 6

A company-specific packet with sourced problems and an artifact plan.

04 PEOPLE
Maps relevant people by proximity

05 TECHNICAL NOTE
Writes one-page problem brief

06 OUTREACH
Drafts email, LinkedIn note, follow-up

07 QA
Checks claims, sources, word count, tone

Subject: FlashAttention + distributed inference scheduling

Hey Tri,

I've been reading about FlashAttention and how it cuts inference latency by rethinking attention memory access – really cool stuff. I'm a student at UConn studying AI and convex optimization, and I've been prototyping a dynamic batching and scheduling agent for distributed LLM inference on a small Kubernetes cluster. The idea is to use convex optimization (inspired by my coursework) to balance latency and throughput by monitoring GPU utilization and queue depth in real time.

Would you be open to a 15-min chat about how FlashAttention changes the scheduling trade-offs for inference at scale?

Luis

LINKEDIN NOTE27%

Loved your work on FlashAttention – huge impact on inference latency. I'm a student building a scheduling agent for distributed LLM inference and would value your perspective.

UPSEARCH

Find the next technical conversation worth having.

Start with a company or a problem space. End with a focused packet and one message you are ready to stand behind.

